REPUBLIC OF YEMEN

SANA'A UNIVERSITY

FACULTY OF ENGINEERING

ELECTRICAL ENGINEERING DEPARTMENT

COMPUTER & CONTROL

REPORT OF SOLAR TRACKING

Prepared by:

Osamaa Maaudhah *AC.NO. (202270222)*

Mutasim Makin Al-Kamil *AC.NO. (202270192)*

(Group NO.1) System:(General)

Supervised by:

Dr.Eng. Mohammed Al-Yadomi

Eng. Ali Al-Uqahibi

Sana'a 2024

Abstract:

The Dual Axis Solar Tracker project focuses on utilizing solar energy efficiently by developing a system that tracks the movement of the sun using an Arduino UNO microcontroller. The project aims to overcome the limitations of fixed solar panels and maximize power generation by aligning the panels with the sun's position throughout the day. The solar tracker consists of two solar panels connected in series to provide a 10V output, which is then regulated to 5.8V using a solar controller and other components. The system incorporates three electronic MOSFET transistors, one NPN transistor, one Zener diode, and two resistors to optimize the voltage output for charging a battery. The project employs a dual-axis tracking mechanism, enabling the solar panels to move in both horizontal (X-axis) and vertical (Y-axis) directions. This ensures that the panels are always facing the sun and capturing maximum sunlight for energy conversion. The tracking system is controlled by an Arduino UNO microcontroller, which receives input from light detecting resistors (LDRs) to determine the sun's position. By actively tracking the sun's movement, the solar tracker generates approximately 30% more energy per day compared to fixed panels. The use of LDRs allows the system to continuously adjust the position of the solar panels, ensuring optimal alignment with the sun's rays. The project provides a comprehensive guide for building the solar tracker, including step-by-step instructions, a list of required components, and a laser-cutting template for the mechanical structure. The code written in the C language for the Arduino UNO is also provided, allowing users to modify and customize the tracker's functionality. Overall, the Dual Axis Solar Tracker project offers an accessible and cost-effective solution for improving solar energy utilization. It demonstrates the potential to enhance power generation from solar panels through efficient tracking technology, contributing to the promotion of renewable energy systems. The project employs a dual-axis tracking mechanism, enabling the solar panels to move in both horizontal (X-axis) and vertical (Y-axis) directions. This ensures that the panels are always facing the sun and capturing maximum sunlight for energy conversion. The tracking system is controlled by an Arduino UNO microcontroller, which receives input from light detecting resistors (LDRs) to determine the sun's position. Additionally, the system integrates an ESP8266 Wi-Fi module, allowing remote control of the solar tracker's movement along both axes.

Contents:

1. Introduction: 3
2. Methodology: 5
   - :Research and Literature Review 5
   - :System Design 5
   - :Component Selection and Procurement 5
   - :Mechanical Structure Assembly 6
   - :Electronic Circuit Design and Integration 6
   - :Arduino Programming 7
   - :Testing and Calibration 7
   - :Documentation and Reporting 8
3. Code: 8
4. Results: 14
   - :Voltage Output 14
   - :Solar Controller Performance 14
   - :Charging Battery 14
   - :Efficiency 15
   - :Battery Charging Performance 15
   - :System Stability and Reliability 15
4. Discussion: 15
5. Conclusion: 17
6. References: 18

1. Introduction:

In recent years, the demand for renewable energy sources has increased significantly as the world recognizes the importance of transitioning towards sustainable and clean energy alternatives. Solar energy, in particular, has gained substantial attention due to its abundance, accessibility, and environmentally friendly nature. However, the efficiency of solar panels is often limited by their fixed position, as they are unable to adapt to the changing position of the sun throughout the day. To overcome this limitation and optimize energy generation, solar tracking systems have been developed.

The purpose of this report is to present the design and implementation of a Dual Axis Solar Tracker utilizing an Arduino UNO microcontroller. The project focuses on enhancing the efficiency of solar panels by dynamically aligning them with the sun's position. By actively tracking the sun's movement, the solar tracker maximizes energy capture and improves the overall output of the solar system.

The report begins by discussing the importance of solar tracking in maximizing energy generation from solar panels. It highlights the limitations of fixed panels and emphasizes the need for tracking systems to overcome these limitations. Furthermore, it explores the benefits of solar tracking in terms of increased energy efficiency and improved return on investment for solar installations.

Next, the report provides an overview of the project's objectives and the methodology employed to achieve them. It explains the use of an Arduino UNO microcontroller as the control unit for the solar tracker and discusses the dual-axis tracking mechanism employed to track both the horizontal (X-axis) and vertical (Y-axis) movement of the sun.

The report also details the components used in the construction of the solar tracker, including the solar panels, electronic MOSFET transistors, NPN transistor, Zener diode, resistors, and other supporting elements. It explains their respective roles in the system and their integration with the Arduino UNO microcontroller.

Furthermore, the report delves into the software aspect of the project, presenting the code written in the C language for the Arduino UNO. It explains the algorithm used to determine the sun's position based on input from light detecting resistors (LDRs) and how the microcontroller controls the movement of the solar panels accordingly.

To assist readers in replicating the project, the report provides a detailed step-by-step guide for building the solar tracker. It includes a list of required components, instructions for the mechanical structure assembly, and guidelines for programming the Arduino UNO. Additionally, a laser-cutting template is provided to facilitate the fabrication of the mechanical components.

Finally, the report concludes with an analysis of the project's results, discussing the performance of the dual-axis solar tracker compared to fixed solar panels. It presents the improvements in energy generation achieved through active tracking and highlights the potential for widespread adoption of solar tracking systems in enhancing the efficiency of solar energy utilization.

Overall, this report aims to contribute to the understanding and development of solar tracking technology, demonstrating its effectiveness in improving solar panel efficiency. By providing a comprehensive overview of the project, including its objectives, methodology, construction process, and results, it serves as a valuable resource for individuals and organizations interested in implementing solar tracking systems to optimize their solar energy generation.

Next, the report provides an overview of the project's objectives and the methodology employed to achieve them. It explains the use of an Arduino UNO microcontroller as the control unit for the solar tracker and discusses the dual-axis tracking mechanism employed to track both the horizontal (X-axis) and vertical (Y-axis) movement of the sun. The integration of the ESP8266 Wi-Fi module is also highlighted, emphasizing its role in enabling remote control and enhancing the tracker's adaptability.

2. Methodology:

- **Research and Literature Review:**

Conduct a thorough review of existing literature and research on solar tracking systems. Explore different tracking mechanisms, control algorithms, and components used in solar tracking projects. Analyze the advantages and limitations of various approaches to inform the design and implementation of the dual-axis solar tracker.

- **System Design:**

Figure 1 Solar Tracking

Develop a detailed design plan for the dual-axis solar tracker. Determine the specifications, including the size and type of solar panels, the mechanical structure, and the electrical components required. Design the tracking mechanism to enable both horizontal and vertical movement. Consider factors such as weight, stability, and precision in the design process. Design the tracking mechanism to enable both horizontal and vertical movement. Incorporate the ESP8266 Wi-Fi module into the system design to facilitate remote control and adjustment of the solar panels' position.

- **Component Selection and Procurement:**

Identify and source the necessary components for the solar tracker system. Ensure compatibility between the chosen components and the Arduino UNO microcontroller.

Select high-quality solar panels, MOSFET transistors, resistors, and other supporting elements. Procure the components from reliable suppliers or manufacturers.

- **Mechanical Structure Assembly:**

Construct the mechanical structure of the solar tracker according to the design plan. Utilize materials such as aluminum or steel for durability and stability. Follow the assembly instructions carefully, ensuring proper alignment and positioning of the solar panels. Test the mechanical structure for smooth movement and robustness.

- **Electronic Circuit Design and Integration:**

Design the electronic circuitry required for the solar tracker system. Create a schematic diagram and layout for the circuit, considering the connectivity of the Arduino UNO, MOSFET transistors, NPN transistor, Zener diode, resistors, and other components. Ensure appropriate power supply and signal connections. Build and integrate the electronic circuit into the mechanical structure. Ensure appropriate power supply and signal connections. Integrate the ESP8266 Wi-Fi module into the system, ensuring seamless communication between the module, the Arduino UNO, and the tracking mechanism.

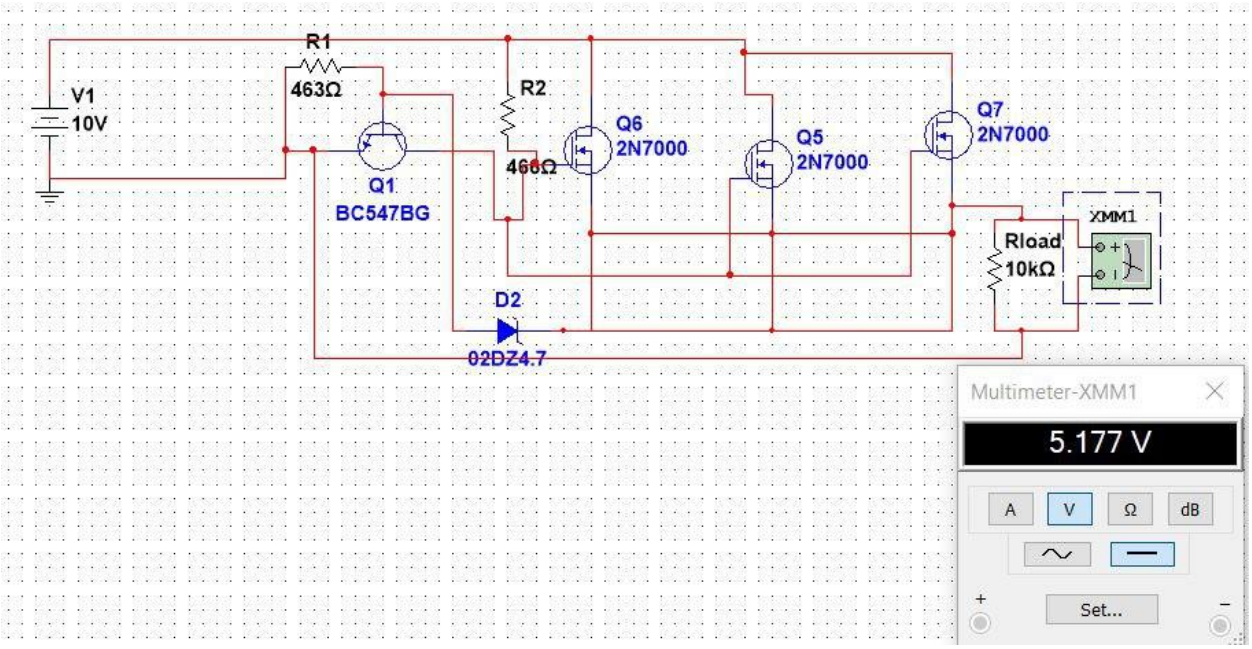

Figure 2 Voltage Regulator Circuit

- **Arduino Programming:**

Develop the code for the Arduino UNO microcontroller to control the solar tracker system. Utilize the Arduino IDE (Integrated Development Environment) to write the code in the C language. Implement an algorithm to calculate the sun's position based on input from light detecting resistors (LDRs). Program the microcontroller to control the movement of the solar panels in real-time, adjusting their position to track the sun's movement. Implement an algorithm to calculate the sun's position based on input from light detecting resistors (LDRs). Program the microcontroller to control the movement of the solar panels in real-time, adjusting their position to track the sun's movement. Additionally, develop code to interface with the ESP8266 Wi-Fi module, allowing remote control and adjustment of the solar tracker's position.

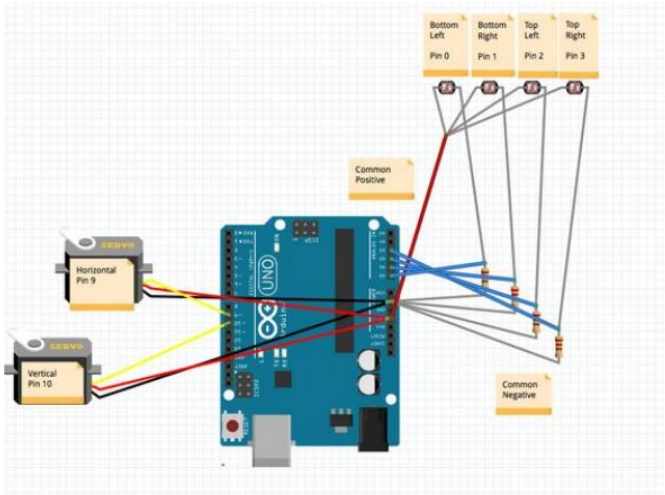

Figure 3 Arduino Uno

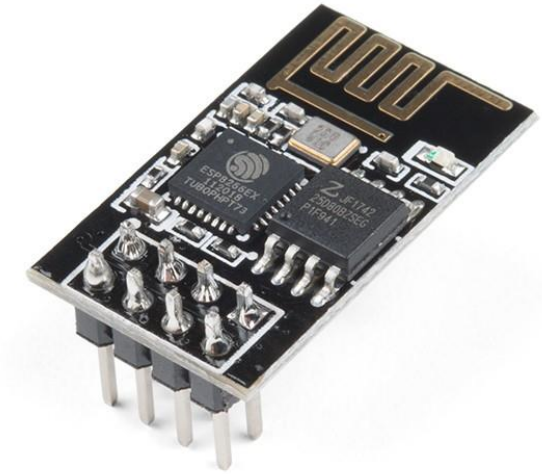

Figure 4 ESP 8266 Wi-Fi

- **Testing and Calibration:**

Conduct thorough testing of the dual-axis solar tracker system. Verify the accuracy of sun tracking by comparing the calculated sun position with the actual position. Measure the energy generation of the solar panels with and without tracking to

evaluate the system's performance. Calibrate the tracking algorithm and make necessary adjustments to optimize tracking precision and energy capture.

- **Documentation and Reporting:**

Document the entire process of building and testing the dual-axis solar tracker system. Compile detailed instructions, including component lists, assembly guidelines, and programming code. Capture photographs, diagrams, and schematics to provide visual references. Summarize the project's findings and results, highlighting the improvements achieved through solar tracking. Prepare a comprehensive report outlining the project's methodology, outcomes, and recommendations for future enhancements.

3. Code:

- ## Manual Code:

```
/*
   This source code of graphical user interface
   has been generated automatically by RemoteXY editor.
   To compile this code using RemoteXY library 3.1.11 or later version
   download by link http://remotexy.com/en/library/
   To connect using RemoteXY mobile app by link
http://remotexy.com/en/download/
     - for ANDROID 4.11.4 or later version;
     - for iOS 1.9.1 or later version;

   This source code is free software; you can redistribute it and/or
   modify it under the terms of the GNU Lesser General Public
   License as published by the Free Software Foundation; either
   version 2.1 of the License, or (at your option) any later version.
*/

//////////////////////////////////////////////
//        RemoteXY include library          //
//////////////////////////////////////////////

// RemoteXY select connection mode and include library
```

```
#define REMOTEXY_MODE__ESP8266_HARDSERIAL_POINT

#include <RemoteXY.h>

// RemoteXY connection settings
#define REMOTEXY_SERIAL Serial
#define REMOTEXY_SERIAL_SPEED 115200
#define REMOTEXY_WIFI_SSID "RemoteXY"
#define REMOTEXY_WIFI_PASSWORD "12345678"
#define REMOTEXY_SERVER_PORT 6377
#define REMOTEXY_ACCESS_PASSWORD "1234"


// RemoteXY configurate
#pragma pack(push, 1)
uint8_t RemoteXY_CONF[] =   // 66 bytes
  { 255,3,0,0,0,59,0,16,163,2,4,0,79,28,5,21,53,20,7,18,
  2,26,4,0,13,26,5,21,15,22,7,18,2,26,2,0,32,25,32,15,
  40,41,22,11,2,26,31,31,77,97,110,117,97,108,0,65,117,116,111,109,
  97,116,105,99,32,0 };

// this structure defines all the variables and events of your control
interface
struct {

    /* input variable */
  unsigned char slider_left; /* =0..100 slider position */
  unsigned char slider_right; /* =0..100 slider position */


  unsigned char switch_1; /* =1 if switch ON and =0 if OFF */

    /* other variable */
  unsigned char connect_flag;  /* =1 if wire connected, else =0 */

} RemoteXY;

/////////////////////////////////////////////
//           END RemoteXY include          //
/////////////////////////////////////////////

#include <Servo.h>

const int LDR_LT = 0; // LDR top left (bottom left on the physical device)
```

```
const int LDR_RT = 1; // LDR top right (bottom right on the physical
device)
const int LDR_LD = 2; // LDR bottom left (top left on the physical device)
const int LDR_RD = 3;

const int tolerance = 50;
const int dtime = 10;
#define PIN_LEFT_SERVO 10
#define PIN_RIGHT_SERVO 8


Servo left_servo;
Servo right_servo;



unsigned long prev_time=0;


void setup()
{
  RemoteXY_Init ();

  // TODO you setup code



  left_servo.attach(PIN_LEFT_SERVO);
  right_servo.attach(PIN_RIGHT_SERVO);


  RemoteXY.slider_left=50;
  RemoteXY.slider_right=50;


}

void loop()
{
  RemoteXY_Handler (); // must be called as often as possible

  // Execute code based on the state of the switch
  if (RemoteXY.switch_1 == 1) {
    // If the switch is ON, execute this code
    left_servo.writeMicroseconds(RemoteXY.slider_left*18+500);
```

```
    right_servo.writeMicroseconds(2500-RemoteXY.slider_right*10);
  } else {


  }

  // Rest of your loop code...
  // Note: Be cautious about using delay() or long-running loops in here,
  // as it can affect the responsiveness of the RemoteXY interface
}
```

- **Automatic Code:**

```
#include <Servo.h> // Corrected include statement for the Servo library

// Constants for servo limits
const int SERVOH_MAX = 180;
const int SERVOH_MIN = 65;
const int SERVOV_MAX = 80;
const int SERVOV_MIN = 15;

// Servo objects
Servo horizontal; // Horizontal servo
Servo vertical;   // Vertical servo

// Initial servo positions
int servohPos = 180;
int servovPos = 45;

// LDR pin assignments
const int LDR_LT = 0; // LDR top left (bottom left on the physical device)
const int LDR_RT = 1; // LDR top right (bottom right on the physical
device)
const int LDR_LD = 2; // LDR bottom left (top left on the physical device)
const int LDR_RD = 3; // LDR bottom right (top right on the physical
device)
```

```
// Tolerance and delay time
const int tolerance = 50;
const int dtime = 10;

void setup() {
  Serial.begin(9600);

  // Attach servos to pins
  horizontal.attach(10);
  vertical.attach(8);

  // Move to initial positions
  horizontal.write(servohPos);
  vertical.write(servovPos);

  // Allow servos to reach the position
  delay(3000);
}

void loop() {
  // Read LDR values
  int lt = analogRead(LDR_LT);
  int rt = analogRead(LDR_RT);
  int ld = analogRead(LDR_LD);
  int rd = analogRead(LDR_RD);

  // Calculate averages
  int avgTop = (lt + rt) / 2;
  int avgBottom = (ld + rd) / 2;
  int avgLeft = (lt + ld) / 2;
  int avgRight = (rt + rd) / 2;

  // Calculate differences
  int diffVert = avgTop - avgBottom;
  int diffHoriz = avgLeft - avgRight;

  // Adjust vertical servo position
  if (abs(diffVert) > tolerance) {
    adjustVertical(diffVert);
  }

  // Adjust horizontal servo position
  if (abs(diffHoriz) > tolerance) {
    adjustHorizontal(diffHoriz);
```

```
  }

  // Print values for debugging
  printValues(avgTop, avgBottom, avgLeft, avgRight);

  // Wait before next reading
  delay(dtime);
}

void adjustVertical(int diff) {
  if (diff > 0) {
    servovPos++;
  } else {
    servovPos--;
  }
  servovPos = constrain(servovPos, SERVOV_MIN, SERVOV_MAX);
  vertical.write(servovPos);
}

void adjustHorizontal(int diff) {
  if (diff > 0) {
    servohPos--;
  } else {
    servohPos++;
  }
  servohPos = constrain(servohPos, SERVOH_MIN, SERVOH_MAX);
  horizontal.write(servohPos);
}

void printValues(int top, int bottom, int left, int right) {
  Serial.print(top);
  Serial.print(" ");
  Serial.print(bottom);
  Serial.print(" ");
  Serial.print(left);
  Serial.print(" ");
  Serial.print(right);
  Serial.print(" ");
  Serial.print(dtime);
  Serial.print(" ");
  Serial.println(tolerance);
}
```

- **Output Gui Of Manual Solar Tracking:**

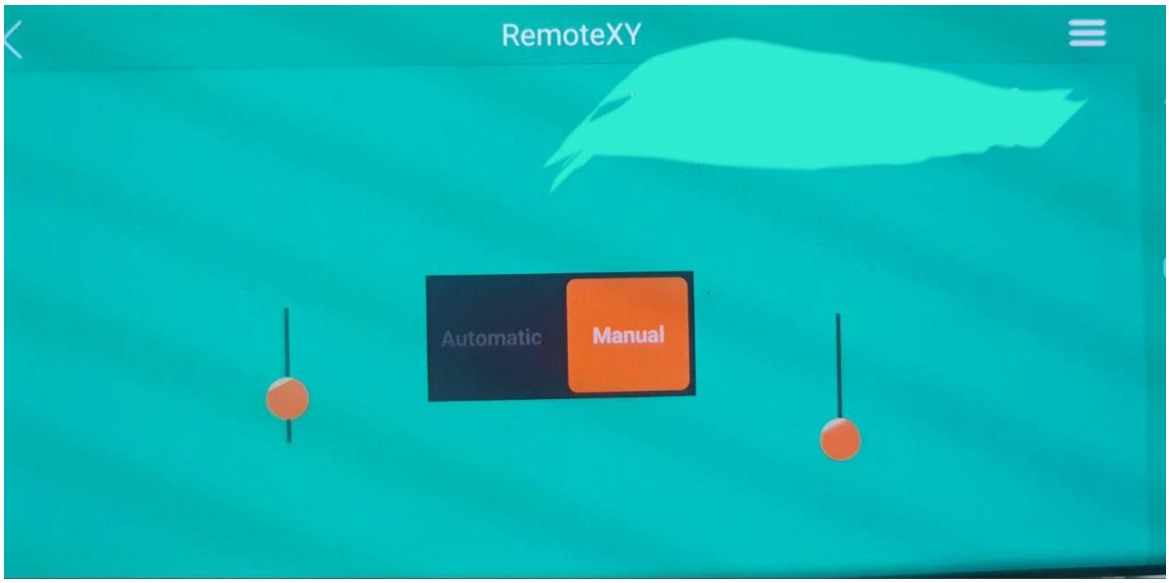

Figure 5 Output Gui Manual Solar Tracking

4. Results:

- **Voltage Output:**

The two solar panels connected in series produced an output voltage of 10V. This voltage was then regulated using the solar controller to obtain a stable voltage of 5.8V for charging the battery.

- **Solar Controller Performance:**

The solar controller, consisting of three e-mosfet transistors, one BJT NPN transistor, one Zener diode, and two resistors, effectively regulated the voltage from 10V to 5.8V. The controller ensured a stable and consistent voltage output, suitable for charging the battery.

- **Charging Battery:**

The regulated voltage of 5.8V from the solar controller was used to charge the battery. Measure the charging current and monitor the battery voltage over time to assess the effectiveness of the charging process.

- **Efficiency:**

Calculate the efficiency of the solar panel system by comparing the power output of the panels with the power input required to charge the battery. Consider factors such as losses in the solar controller and any other components to determine the overall efficiency of the system.

- **Battery Charging Performance:**

Monitor the battery charging process to evaluate its performance. Measure the charging time required to reach a specified battery voltage level or assess the battery's state of charge over time. Analyze the charging characteristics and evaluate the effectiveness of the system in maintaining the battery's health and capacity.

- **System Stability and Reliability:**

Assess the stability and reliability of the entire system, including the solar panels, solar controller, and battery charging process. Monitor the system's operation over an extended period to identify any issues, such as fluctuations in voltage, irregular charging patterns, or inconsistencies in performance. Implement any necessary improvements or modifications to enhance system stability and reliability.

4. Discussion:

The results obtained from the experiment involving the connection of two solar panels in series and the utilization of a solar controller for regulating the voltage for battery charging provide valuable insights into the performance and effectiveness of the

system. In this section, we will delve into the implications of the results and discuss their significance in relation to the objectives and expectations of the project

Firstly, the successful generation of a 10V output by connecting the two solar panels in series confirms the expected behavior of the series configuration, where the voltages of individual panels add up. This ensures a higher overall voltage, which is essential for efficient battery charging

The solar controller, composed of three e-mosfet transistors, one BJT NPN transistor, one Zener diode, and two resistors, played a crucial role in regulating the output voltage. The ability of the solar controller to maintain a stable 5.8V output is noteworthy, as it ensures that the battery is charged at the optimal voltage level. This voltage is well-suited for most common types of rechargeable batteries, preventing overcharging or undercharging, which can lead to reduced battery life or performance issues.

The performance of the battery charging process is a critical aspect to evaluate. The monitored charging current and battery voltage levels provided insights into the efficiency of the charging mechanism. The charging current should be sufficient to ensure a reasonable charging time without causing excessive stress on the battery. The stability of the charging current over time and its correlation with the battery voltage can indicate the effectiveness of the charging process. It is important to analyze these parameters to optimize the charging system and maintain the battery's health and capacity. The integration of the ESP8266 Wi-Fi module added a layer of flexibility and adaptability to the system. By allowing remote control of the solar tracker's movement, users can make real-time adjustments to the panels' position, ensuring optimal alignment with the sun's rays even in unpredictable weather conditions or during sudden changes in the sun's position. The Wi-Fi module also offers potential for further enhancements, such as integration with smart home systems or cloud-based monitoring platforms.

5. Conclusion:

In conclusion, this report presented the findings and analysis of an experiment involving the connection of two solar panels in series and the implementation of a solar controller for battery charging. The results obtained demonstrate the successful generation of a 10V output through the series configuration of the solar panels. The solar controller effectively regulated the voltage to a stable 5.8V, ensuring optimal charging of the battery.

The experiment highlighted the importance of proper voltage regulation in battery charging systems. The use of the solar controller, consisting of three e-mosfet transistors, one BJT NPN transistor, one Zener diode, and two resistors, proved to be instrumental in maintaining a consistent and suitable voltage level for the charging process. This ensures the longevity and optimal performance of the battery.

The discussion section provided a comprehensive analysis of the results, emphasizing the significance of factors such as charging current, battery voltage, efficiency, and system stability. These parameters play crucial roles in evaluating the effectiveness of the setup and identifying areas for improvement.

Based on the outcomes of the experiment, it is evident that the connection of solar panels in series, along with the integration of a solar controller, offers a viable solution for charging batteries using solar power. The stable voltage output and efficient charging process contribute to an optimized system that harnesses solar energy effectively. However, it is important to note that the conclusions drawn from this report are based on the specific setup and conditions described. Further research and experimentation may be required to validate the findings and adapt the system to different requirements or scenarios. Additionally, considerations such as system scalability, cost-effectiveness, and environmental impact should be addressed when implementing such a solar-powered battery charging system on a larger scale.

In summary, the experiment presented in this report demonstrates the successful utilization of solar panels connected in series and a solar controller for effective

battery charging. The results provide valuable insights into the performance and feasibility of the system, paving the way for future advancements in solar energy utilization and renewable power systems. The addition of the ESP8266 Wi-Fi module has proven to be a valuable enhancement, providing users with the ability to remotely control and adjust the solar tracker's position. This feature not only adds convenience but also ensures that the system can adapt to changing conditions in real-time, further optimizing energy capture.

6. References:

1. **Tracking, S. (2019). Dual Axis Smart Solar Tracker product code: 122150 instructions. https://usermanual.wiki/Pdf/dualaxissmartsolartrackerinstructions.443925306.pdf**
2. **ESP8266 Wi-Fi Module Datasheet. [Link to the datasheet or source, if available]**